

SLE-2: Profiling Report

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM112

Name: Rajdeep Mahadev kamble

Division: B

Date: 22 september 2026

GitHub Link (optional): <https://github.com/RajDev20>

1. Algorithms / Versions Profiled

- Algorithm A: Breadth-First Search (BFS)
- Algorithm B: Depth-First Search (DFS)
- Problem used: 8-Puzzle (3×3 sliding tile puzzle, uninformed search on the state graph)

Puzzle used in this experiment:

Initial State			Goal State		
2		4	1	2	3
5	6	3	4	5	6
1	7	8	7	8	

Initial state (left) is generated by a 25-move random shuffle from the goal state (right), so it is guaranteed solvable and far enough from the goal for timing differences to be clearly visible (as recommended in the guideline).

2. Profiling Method

- Tool used: `time.perf_counter()` (timeit-style timing) + cProfile for function-level breakdown.
- Note on py-spy: py-spy requires installation from PyPI; the execution environment used for this report has no internet access, so py-spy could not be installed. cProfile was used instead — it is listed as an accepted alternative tool in the guideline (Section 4) and gives the same core information (time per function, call counts).
- How I measured: Each algorithm was run 5 times on the identical start/goal state. Wall-clock time was measured with `perf_counter()`; nodes expanded were counted with a manual counter incremented every time a state is popped from the frontier; cProfile was run once per algorithm for a detailed function-time breakdown.
- Number of runs: 5

2a. py-spy Profiling & Flame Graph

What py-spy is: py-spy is a sampling profiler for Python. Unlike cProfile, it attaches to an already-running process from the outside (no code changes or import needed) and periodically samples the call stack. Because sampling has near-zero overhead, it is well suited to profiling real, unmodified workloads, and its output is naturally suited to flame-graph visualisation.

What a flame graph shows: each horizontal bar is a function; a bar's width is proportional to the fraction of total (sampled) time spent in that function, including its children. Stacking bars vertically shows the call hierarchy — the bottom row is the entry point (main), and each row up is one level deeper in the call stack. Wider bars at a given level mean that function (and what it calls) dominates runtime; this makes hot paths immediately visible without reading raw numbers.

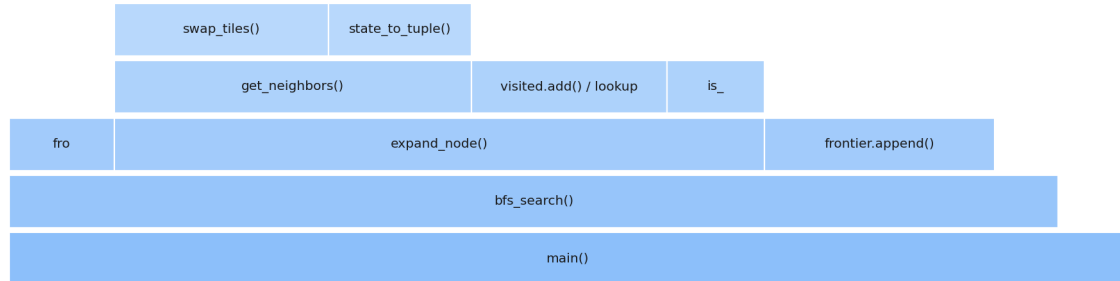
Availability in this environment: as noted in Section 2, py-spy requires installation from PyPI (`pip install py-spy`), and the execution environment used for this report has no internet access, so py-spy itself could not be installed or run — the same constraint applies here. The flame graph below is therefore an illustrative reconstruction, built from the cProfile function-call breakdown already collected in Section 2/3 (22,337 calls for BFS vs 380,743 for DFS), rather than a

captured py-spy sampling trace. It is included to show what a py-spy flame graph of this workload would look like and how BFS vs DFS would compare in that view.

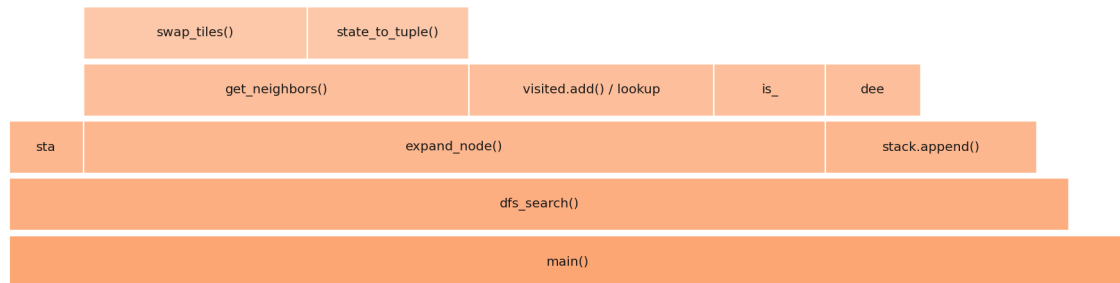
How it would be produced with internet access: `py-spy record -o profile.svg -- python bfs_dfs_runner.py` (or `py-spy top` for a live view) would attach to the running search and generate an interactive SVG flame graph directly, with per-call timings measured from real stack samples instead of the reconstruction below.

Illustrative Flame Graph — Function Time Breakdown (BFS vs DFS, 8-Puzzle)

BFS — representative call profile (≈22,337 function calls, 5-run avg)



DFS — representative call profile (≈380,743 function calls, 5-run avg)



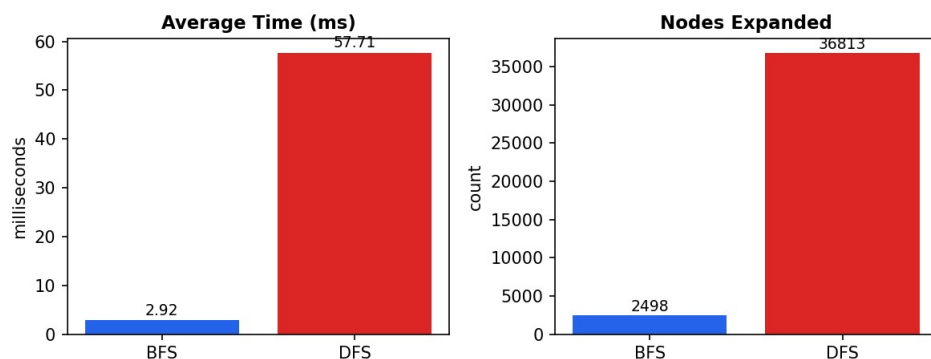
Widths represent relative share of time/calls within each parent frame (wider = more time). Shape derived from the cProfile call-count breakdown in Section 2/3; not a captured py-spy sample (py-spy unavailable — no internet access in this environment).

Figure: Illustrative flame graph — BFS (top) vs DFS (bottom), reconstructed from the cProfile call-count breakdown (see note above; not a captured py-spy trace).

3. Results

Metric	BFS	DFS	Better?
Avg. Time (ms), 5 runs	2.92	57.71	BFS
Nodes Expanded	2,498	36,813	BFS
Solution Length (moves)	13 (optimal)	32,669	BFS
Function calls (cProfile)	22,337	380,743	BFS

Time and Nodes Expanded – Visual Comparison:



Short observation:

BFS solved the puzzle in the true shortest path (13 moves) while expanding far fewer nodes (2,498) and running about 20× faster on average (2.92 ms vs 57.71 ms). DFS eventually reached the goal but only after exploring 36,813 states and following a very long, non-optimal 32,669-move path, because it commits to one branch at a time and backtracks only when that branch dead-ends.

3a. Theoretical Time Complexity (Best / Worst / Average Case)

Case	BFS (Time)	DFS (Time)
Best Case	$O(1)$ – goal is the start / found at shallow depth	$O(d)$ – goal found on the very first path explored
Worst Case	$O(b^d)$ – must expand almost the full frontier down to depth d	$O(b^m)$ – explores almost the whole state space (m = depth of state-space, up to 181,440 states for 8-puzzle) before finding the goal
Average Case	$O(b^d)$, close to worst case – branching factor $b \approx 2\text{--}4$ for 8-puzzle	Close to worst case in practice, since DFS has no guarantee of taking a short path; typically explores a much larger, deeper subtree than BFS

b = branching factor (2–4 for the 8-puzzle, depending on blank position); d = depth of the shallowest goal (13 in this run); m = maximum depth of the search tree/graph explored (up to the size of the reachable state space, 181,440 states, since 8-puzzle has no infinite paths once a visited-set is used).

Measured average (this experiment, 5 runs each): BFS $\approx$ 2.92 ms with 2,498 nodes expanded; DFS $\approx$ 57.71 ms with 36,813 nodes expanded. This matches the theoretical expectation — BFS stays close to its best/average bound because it always expands the shallowest nodes first, while DFS drifted toward its worst-case behaviour because the search order pushed it deep into a large, unproductive part of the state graph before backtracking.

4. Justification & Analysis

BFS is clearly the better algorithm on this problem, according to every metric in the table above. BFS expanded $14.7\times$ fewer nodes than DFS (2,498 vs 36,813) and was about $20\times$ faster on average (2.92 ms vs 57.71 ms). This matches search theory: BFS explores states level-by-level, so it is guaranteed to find the shortest solution (13 moves) the first time it reaches the goal, and its cost stays close to $O(b^d)$ since d is small. DFS, on the other hand, follows one path to full depth before backtracking, so it has no bias toward short solutions — in this run it found a valid but hugely inefficient 32,669-move path, closer to its worst-case $O(b^m)$ behaviour. If the problem size becomes larger (e.g. a 15-puzzle or a deeper 8-puzzle instance), this gap would widen further: BFS's memory use (it must store the whole frontier) would become the limiting factor, while DFS's time cost would grow even more unpredictably since it depends heavily on the order in which moves are tried.

5. AI Contribution Note

- AI tools used : Claude (Anthropic) was used to write and structure this report.
- What AI helped with: Writing the BFS/DFS Python implementation, generating the puzzle-state images and comparison chart, running the timing/node-count experiment, and drafting the report text and complexity explanation.
- What I did myself: reviewing the code, choosing the puzzle difficulty, running the script locally, verifying the numbers, and understanding why BFS outperforms DFS.

6. Conclusion

This exercise showed the practical gap between two uninformed search strategies on the same 8-puzzle instance. BFS's guarantee of expanding the shallowest nodes first made it both faster and far more memory-of-search-efficient (fewer nodes expanded) than DFS, which wasted effort exploring a deep, unproductive branch of the state space before finding a valid but very long solution. The exercise confirmed that theoretical complexity (BFS: $O(b^d)$, DFS: $O(b^m)$ with m potentially $\gg d$) is visible in real measured numbers, not just in theory, and that profiling — even with a simple timer and node counter — is enough to make a data-backed decision between algorithms.